

二、LlamaIndex基础组件介绍

--楼兰

章节目标：

- 理清RAG的工作流程
- 熟悉LlamaIndex的主要组件

一、LlamaIndex与RAG

先来回顾一下上个章节最后LlamaIndex的案例：

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from langchain_community.chat_models import ChatTongyi
from config.load_key import load_key
from llama_index.embeddings.dashscope import (
    DashScopeEmbedding,
    DashScopeTextEmbeddingModels,
    DashScopeTextEmbeddingType,
)
from pydantic.types import SecretStr

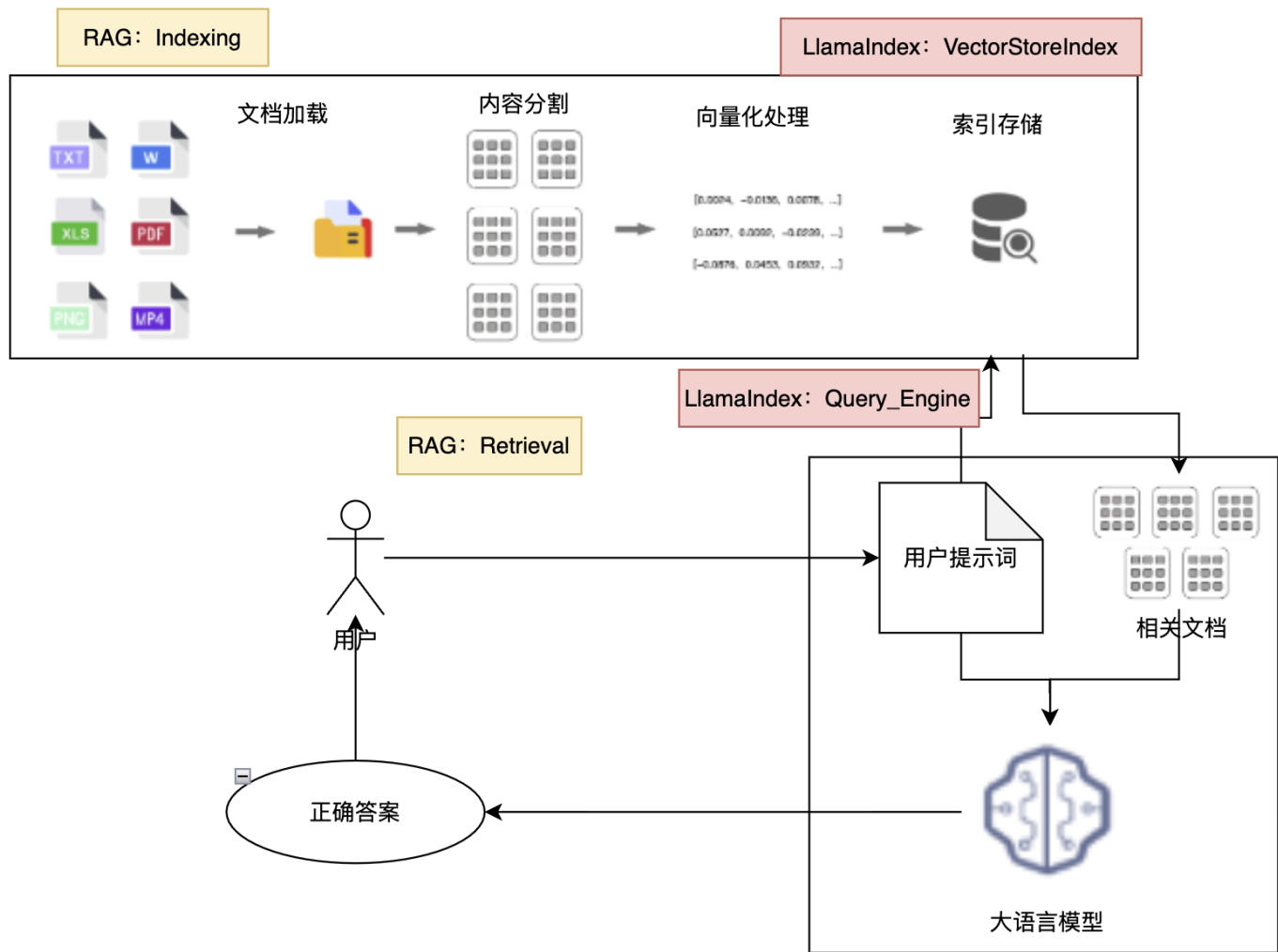
# 初始化通义千问的Embedding模型
Settings.embed_model=DashScopeEmbedding(
    model_name=DashScopeTextEmbeddingModels.TEXT_EMBEDDING_V2,
    text_type=DashScopeTextEmbeddingType.TEXT_TYPE_DOCUMENT,
    api_key=load_key("BAILIAN_API_KEY"),
)

Settings.llm=ChatTongyi(
    model="qwen-plus",
    api_key=SecretStr(load_key("BAILIAN_API_KEY")),
)

#加载参考数据
documents = SimpleDirectoryReader("resource").load_data()
index = VectorStoreIndex.from_documents(documents)

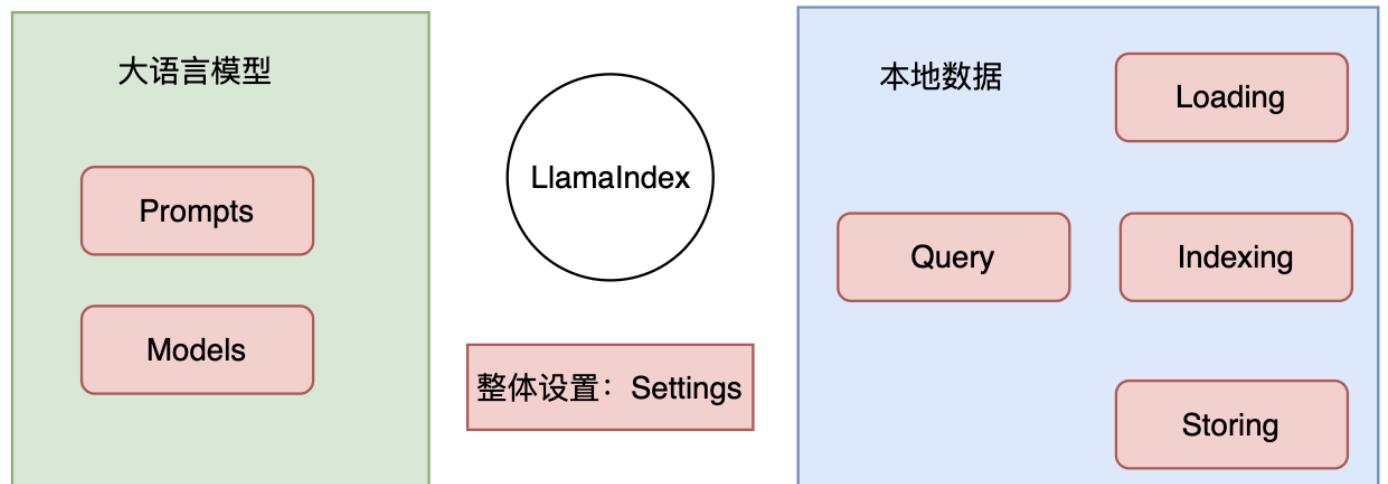
# 构建检索引擎
query_engine = index.as_query_engine()
response = query_engine.query("怎么退款")
response
```

这个案例中可以看到，LlamaIndex的核心思想就是在用户询问大语言模型时，可以让大语言模型先从本地文件中检索出最相关的内容，然后把这些内容和用户问题一起发给大语言模型，让大语言模型基于这些内容进行回答。这样可以极大提高大语言模型的回答质量。



二、LlamaIndex基础组件介绍

在具体实现时，LlamaIndex采用组件化的方式，将大语言模型、向量数据库、索引、数据源等组件进行组合，实现大语言模型与数据源的交互。通过组件化的方式，可以像拼积木一样，将不同的组件进行组合，实现更好的可扩展性。这里就介绍LlamaIndex中几个重要的组件。



1、Prompts 构建提示词

在和大语言模型交互的过程中，提示词是最为重要的工具。将对提示词的设计和处理封装成几个重要的参数，这也是构建AI应用最简单的一种方式。LlamaIndex提供了很方便的组件，可以用来构建提示词。

```
from llama_index.core import PromptTemplate

template = (
    "把语句 \"{text}\" 翻译成 {language}"
)
# 构建提示词模板
prompt_template = PromptTemplate(template=template)

# 将提示词模板格式化为具体提示词
prompt = prompt_template.format(text="it is such a rainy day", language="中文")
print(prompt)

# 将提示词模板格式化为聊天消息
messages = prompt_template.format_messages(text="it is such a rainy day", language="中文")
messages
```

除了可以对单条提示词进行格式化，还可以通过ChatPromptTemplate同时格式化多条消息记录。

```
from llama_index.core import ChatPromptTemplate
from llama_index.core.llms import ChatMessage, MessageRole

# 消息记录
message_templates = [
    ChatMessage(content="你是一个专业的语言翻译，负责将用户的语句翻译成 {language}。",
role=MessageRole.SYSTEM),
    ChatMessage(
        content="{text}",
        role=MessageRole.USER,
    ),
]
# 构建模板
chat_template = ChatPromptTemplate(message_templates=message_templates)
# 格式化为文本
prompt = chat_template.format(text="it is such a rainy day", language="中文")
print(prompt)
# 格式化为消息
messages = chat_template.format_messages(text="it is such a rainy day", language="中文")
print(messages)
```

另外，LlamaIndex还提供了一个RichPromptTemplate，也可以用来动态构建提示词。

```

from llama_index.core.prompts import RichPromptTemplate

prompt_tmpl = RichPromptTemplate(
    "把用户的问题: \"{{ text }}\" 翻译成 {{ language }}",
)

prompt_str = prompt_tmpl.format(text="it is such a rainy day", language="中文")
prompt_str

```

这个RichPromptTemplate是LlamaIndex提供的一种新的构建提示词的工具。这个工具不光可以替换变量，还支持接入jinja2模板，提供更复杂的构建提示词的功能。例如在模板里实现if、eles、for等复杂逻辑。

实际上，LlamaIndex内部也大量运用了提示词。大部分的组件都支持通过get_prompts方法获取当前组件使用的提示词。而且，还允许用户自己修改内部使用的提示词。

```

prompt_dicts = query_engine.get_prompts()
for key in prompt_dicts:
    print(key + " >>> ")
    print(prompt_dicts[key])
    print("=====")

```

这其中，LlamaIndex会使用 text_qa_template 提示词获得一个初始化的答案。并在需要优化答案时，使用 refine_template 对答案进行进一步优化。

```

qa_query_engine = index.as_query_engine()

qa_prompt_tmpl_str = (
    "以下是参考信息.\n"
    "-----\n"
    "{{ context_str }}\n"
    "-----\n"
    "根据参考信息回答用户的问题。回答问题时，模拟郭德纲的语气，让答案幽默一点.\n"
    "问题: {{ query_str }}\n"
    "答案: "
)

qa_prompt_tmpl = RichPromptTemplate(qa_prompt_tmpl_str)

qa_query_engine.update_prompts(
    {"response_synthesizer:text_qa_template": qa_prompt_tmpl}
)

answer = qa_query_engine.query("怎么退款")
answer

```

关于答案优化。可以在构建索引时指定response_mode响应模式。

```

# 构建检索引擎 response_mode 默认值: compact:压缩。 refine: 优化。初始化响应结果会进入优化模板的
existing_answer 参数，进行重新优化。
refine_query_engine = index.as_query_engine(
    response_mode="refine",

```

```

)
refine_prompt_template = (
    "把下面的原始答案，模拟郭德纲的语气，修改得更幽默一点。.\n"
    "-----\n"
    "{{ existing_answer }}\n"
    "-----\n"
    "答案： "
)
refine_template = RichPromptTemplate(refine_prompt_template)
refine_query_engine.update_prompts(
    {"response_synthesizer:refine_template": refine_template}
)
new_response = refine_query_engine.query("怎么退款")
new_response

```

2、Models 大语言模型

LlamaIndex通过抽象出LLM和Embedding类型，来实现与不同大语言模型以及向量化模型的兼容性。而针对不同大语言模型的具体实现，都被封装到单独的依赖库中。具体参见官网：https://docs.llamaindex.ai/en/stable/module_guides/models/llms/modules/

这些不同的依赖库用法大都差不多。以阿里云百炼的Dashscope为例。需要引入对应的依赖库

```

# 引入dashscope的依赖库
!pip install llama-index-llms-dashscope
!pip install dashscope
# 向量化模型库
!pip install llama-index-embeddings-dashscope

```

先来看如何使用Embedding向量化模型：

```

from llama_index.embeddings.dashscope import (
    DashScopeEmbedding,
    DashScopeTextEmbeddingModels,
    DashScopeTextEmbeddingType,
)
from config.load_key import load_key

# 使用Dashscope 的Embedding向量化模型
embedder = DashScopeEmbedding(
    model_name=DashScopeTextEmbeddingModels.TEXT_EMBEDDING_V2,
    text_type=DashScopeTextEmbeddingType.TEXT_TYPE_DOCUMENT,
    api_key=load_key("BAILIAN_API_KEY"),
)
text_to_embedding = ["风急天高猿啸哀", "渚清沙白鸟飞回", "无边落木萧萧下", "不尽长江滚滚来"]
# Call text Embedding
result_embeddings = embedder.get_text_embedding_batch(text_to_embedding)
# requests and embedding result index is correspond to.
for index, embedding in enumerate(result_embeddings):

```

```

if embedding is None: # if the correspondence request is embedding failed.
    print("The %s embedding failed." % text_to_embedding[index])
else:
    print("Dimension of embeddings: %s" % len(embedding))
    print(
        "Input: %s, embedding is: %s"
        % (text_to_embedding[index], embedding[:5])
    )

```

有了这些向量化结果，就可以通过向量相似度计算文本之间的语义相似度。

```

!pip install scikit-learn
!pip install numpy

```

```

from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
text1 = "我喜欢吃苹果"
text2 = "苹果是我最爱吃的水果"
text3 = "今天天气不错"

vector1 = np.array(embedder.get_text_embedding(text1)).reshape(1, -1)
vector2 = np.array(embedder.get_text_embedding(text2)).reshape(1, -1)
vector3 = np.array(embedder.get_text_embedding(text3)).reshape(1, -1)

# 计算余弦相似度
similarity12 = cosine_similarity(vector1, vector2)[0][0]
similarity13 = cosine_similarity(vector1, vector3)[0][0]
print(f"\n{text1}\n 与 \n{text2}\n 相似度: {similarity12:.4f}")
print(f"\n{text1}\n 与 \n{text3}\n 相似度: {similarity13:.4f}")

```

这个案例中，采用余弦相似度，就能计算出不同文本之间的语义相似度。余弦相似度计算出来的语义相似度是一个介于(0,1)之间的值。值越大表示两个文本语义越相似。而这种以向量计算的方式来理解自然语言语义是否相似的方式，也是现在大语言模型理解人类自然语言的基础。

接下来，我们来看看如何使用大语言模型。

```

from config.load_key import load_key
from llama_index.llms.dashscope import DashScope, DashScopeGenerationModels

dashscope_llm =
DashScope(model_name=DashScopeGenerationModels.QWEN_MAX, api_key=load_key("BAILIAN_API_KEY"))
# 直接完成一次交互
resp = dashscope_llm.complete("你是谁？你能帮我解决什么问题？")
print(resp)

```

```
# 流式输出
responses = dashscope_llm.stream_complete("你是谁? 你能帮我解决什么问题? ")
for response in responses:
    print(response.delta, end="\n")
```

如果需要进行多轮对话，就需要通过chat方法同时传入多条消息。

```
from llama_index.core.base.llms.types import MessageRole, ChatMessage

messages = [
    ChatMessage(
        role=MessageRole.SYSTEM, content="你是一位旅行家，擅长介绍每个城市的旅游景点"
    ),
    ChatMessage(role=MessageRole.USER, content="长沙"),
]
resp = dashscope_llm.chat(messages)
print(resp)
# 流式输出
# dashscope_llm.stream_chat(messages)
```

LlamaIndex针对不同的大语言模型，提供了不同的依赖。如果你觉得记不住这么多依赖的话，也可以直接使用LlamaIndex提供的LangChain依赖库。基于LangChain的强大能力，可以更方便的扩展LlamaIndex接入的大模型范围。

具体可以参见上一章节的案例。

3、Loading 文档加载组件

LlamaIndex处理本地数据最基本的方式就是把本地文档加载成Document，再把文档内容拆分成Node。这些Node就是LlamaIndex处理本地数据的基本单位。

LlamaIndex中的Document可以代表一个本地的资源文件。并且不光是文本文件，图片、音频、视频等资源文件都可以作为Document。

```
from llama_index.core import Document
```

```
text_list = ["Q: 在线支付取消订单后钱怎么返还? 订单取消后, 款项会在一个工作日内, 直接返还到您的美团账户余额。Q: 怎么查看退款是否成功? 退款会在一个工作日之内到美团账户余额, 可在“账号管理—我的账号”中查看是否到账。Q: 余额提现到账时间是多久? 1-7个工作日内可退回您的支付账户。由于银行处理可能有延迟, 具体以账户的到账时间为准。",  
"Q: 申请退款后, 商家拒绝了怎么办? 申请退款后, 如果商家拒绝, 此时回到订单页面点击“退款申诉”, 美团客服介入处理。Q: 怎么取消退款呢? 请在订单页点击“不退款了”, 商家还会正常送餐的。Q: 前面下了一个在线支付的单子, 由于未付款, 订单自动取消了, 这单会计算我的参与活动次数吗? 不会。如果是未支付的在线支付订单, 可以先将订单取消(如果不取消需要15分钟后系统自动取消), 订单无效后, 此时您再下单仍会享受活动的优惠。",  
"Q: 为什么我用微信订餐, 却无法使用在线支付? 目前只有网页版和美团外卖手机App(非美团手机客户端)订餐, 才能使用在线支付, 请更换到网页版和美团外卖手机App下单。Q: 如何进行付款? 美团外卖现在支持货到付款与在线支付, 其中微信版与手机触屏版暂不支持在线支付。Q: 如何查看可以在线支付的商家? 你可以在商家列表页寻找带有“付”标识的商家, 提交订单时可以选择支付方式。"]  
documents = [Document(text=t) for t in text_list]  
documents
```

一个Document代表的是一个本地资源文件。在对信息做处理时, 则需要把Document拆分成Node。Node就是LlamaIndex处理本地数据的基本单位。

这时候, 如何把Document拆分成Node呢? LlamaIndex提供了多种NodeParser的具体实现类, 用来按照不同的规则拆分Document。

```
from llama_index.core.node_parser import TokenTextSplitter  
#如果文本的token长度小于chunk_size, 就不会进行拆分。否则, 按照separator进行拆分  
#具体长度的划分方式跟指定的tokenizer有关。默认使用的是tiktoken的tokenizer, 可以自行指定  
# import llama_index.core  
# tokenizer = llama_index.core.get_tokenizer()  
# print(tokenizer)  
  
parser = TokenTextSplitter(separator="Q: ", chunk_size=90)  
  
nodes = parser.get_nodes_from_documents(documents)  
# 输出结果  
for i, node in enumerate(nodes):  
    print(f"Node {i+1}: \n{node.text.strip()} \n")
```

为了简化Document和Node的加载过程, LlamaIndex提供了一个非常简单易用的工具类SimpleDirectoryReader。用于从本地目录加载文档。SimpleDirectoryReader能够处理非常多常见的文档, 包括csv\docx\md\pdf\png\pptx等多种文档内容。

```
from llama_index.core import SimpleDirectoryReader  
  
reader = SimpleDirectoryReader(input_dir="./json1")  
documents = reader.load_data()  
documents
```

大部分常用的文档, 比如txt\pdf\docx等, 都可以通过SimpleDirectoryReader加载。SimpleDirectoryReader还提供了很多扩展属性, 可以加载非常多的文件。

但是如果遇到一些特殊的文档，也可以通过LlamaIndex的设计扩展出来。例如对于常用的JSON文件，或者JSONL文件，SimpleDirectoryReader只能加载出一个文档。但是使用JSONReader可以解析JSON文件，按照指定的格式加载出多个文档。

加载扩展依赖

```
!pip install llama-index-readers-json
```

```
from llama_index.readers.json import JSONReader

# Initialize JSONReader
reader = JSONReader(
    # The number of levels to go back in the JSON tree. Set to 0 to traverse all levels.
    # Default is None.
    # levels_back="<Levels Back>",
    # The maximum number of characters a JSON fragment would be collapsed in the output.
    # Default is None.
    # collapse_length="<Collapse Length>",
    # If True, ensures that the output is ASCII-encoded. Default is False.
    # ensure_ascii="<Ensure ASCII>",
    # If True, indicates that the file is in JSONL (JSON Lines) format. Default is False.
    is_jsonl=True,
    # If True, removes lines containing only formatting from the output. Default is True.
    # clean_json="<Clean JSON>",
)

# Load data from JSON file
documents = reader.load_data(input_file="./jsonl/encyclopedia.jsonl", extra_info={})
documents
```

4、Indexing 索引组件

Indexing是一种数据结构，允许用户从Indexing中快速检索感兴趣的数据。

在LlamaIndex中，Index通常可以由一系列的Document构建出来，然后再构建出一个Query Engine或者Chat Engine。用户就可以基于这些Engine快速检索出感兴趣的数据。

在Index内部，则会以Node的形式保存数据。Node则是Document拆分出来的一个片段。另外，Index也会暴露出一个Retriever接口，进一步支持文件检索。

通常用得更多的是VectorStoreIndex，存储自然语言处理后的向量数据。

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.embeddings.langchain import LangchainEmbedding
from langchain_community.embeddings import DashScopeEmbeddings
from config.load_key import load_key

documents = SimpleDirectoryReader("./resource").load_data()

# 初始化通义千问的Embedding模型
# 这里是将LangChain针对Dashscope实现的DashScopeEmbedding封装成LlamaIndex可以接受的
LangchainEmbedding
embed_model = LangchainEmbedding(
```

```

DashScopeEmbeddings(
    dashscope_api_key=load_key("BAILIAN_API_KEY"),
    model="text-embedding-v1" # 通义千问的Embedding模型名称 默认值
)

index = VectorStoreIndex.from_documents(documents, embed_model=embed_model)
retriever = index.as_retriever()
responses = retriever.retrieve("怎么退款")
for response in responses:
    print(response.node.text)

```

Indexing索引在构建之后，也支持进行持久化保存。这样，下次再次使用这个索引时，就可以从持久化文件中加载索引，从而提高索引的加载速度。

```

from llama_index.core.storage.storage_context import StorageContext
from llama_index.core import load_index_from_storage
index.storage_context.persist(persist_dir="./index")
storage_context = StorageContext.from_defaults(persist_dir="./index")
new_index =
load_index_from_storage(storage_context=storage_context, embed_model=embed_model)
new_retriever = new_index.as_retriever()
new_responses = new_retriever.retrieve("怎么退款")
for new_response in new_responses:
    print(new_response.node.text)

```

另外，在LlamaIndex中，实际上提供了非常多的Indexing组件，可以用来检索不同的数据。例如直接管理SQL数据，实现自然语言检索等。

具体参考官网：https://docs.llamaindex.ai/en/stable/module_guides/indexing/modules/

5、Storing 数据存储组件

把数据检索出来之后，当然就是需要存储了。LlamaIndex提供了非常多的数据存储组件，允许用户定制外部数据存储。

- Document stores: 用来存储Document和Node。
- Index stores: 用来存储index相关的元数据。
- Vector stores: 用来存储向量数据。
- Chat stores: 用来存储聊天记录。
- Property stores: 用来存储知识图谱相关的数据。

其中，用的最多的，通常就是Vector stores，向量存储了。

在LlamaIndex中，支持多种外部向量数据库来存储向量。具体参见官网 https://docs.llamaindex.ai/en/stable/module_guides/storing/vector_stores/

这里以常见的Redis为例。需要注意的是，Redis需要stack插件支持，才能作为向量数据库使用。stack在Redis7版本中需要额外安装，在Redis8中则默认集成了stack。

LlamaIndex中要使用Redis作为向量存储，需要先添加对应的依赖库。

```
!pip install llama-index-vector-stores-redis
```

然后就可以使用Redis作为向量存储，把向量存储到Redis当中。

```
from llama_index.core import VectorStoreIndex
from llama_index.vector_stores.redis import RedisVectorStore

from llama_index.core import StorageContext
from redis import Redis

# create a Redis client connection
redis_client = Redis.from_url("redis://localhost:6379")

# create the vector store wrapper
vector_store = RedisVectorStore(redis_client=redis_client, overwrite=True)

# load storage context
storage_context = StorageContext.from_defaults(vector_store=vector_store)

# build and load index from documents and storage context
index = VectorStoreIndex.from_documents(
    documents, storage_context=storage_context, embed_model=embed_model
)

# build index from vector store
# index = VectorStoreIndex.from_vector_store(vector_store=vector_store)

retriever = index.as_retriever()
responses = retriever.retrieve("怎么退款")
for response in responses:
    print(response.node.text)
```

通过这个案例，我们就可以把检索出来的文本存储到Redis中。

另外还有其他几种Storing组件，在最常见的RAG场景中用得不多。这里就不再多做介绍。

6、Querying 查询引擎

查询引擎是LlamaIndex中最为核心的组件，因为他是直接暴露给用户的，用户可以基于查询引擎进行查询。LlamaIndex中同样也提供了非常多的Querying组件，用来支持不同的查询场景。

例如之前使用的Retriever就是一种Querying查询引擎。他的作用主要是检索出和问题最相关的文档。但是Retriever并不直接回答问题。如果需要回答问题，就可以进一步的封装成Query Engine。

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.llms.langchain import LangChainLLM
from llama_index.embeddings.langchain import LangchainEmbedding
from langchain_community.embeddings import DashScopeEmbeddings
```

```

from langchain_community.chat_models import ChatTongyi
from config.load_key import load_key

llm = ChatTongyi(
    model="qwen-plus",
    api_key=load_key("BAILIAN_API_KEY"),
)
# 初始化通义千问的Embedding模型
embed_model = DashScopeEmbeddings(
    dashscope_api_key=load_key("BAILIAN_API_KEY"),
    model="text-embedding-v1" # 通义千问的Embedding模型名称 默认值
)

# 加载参考数据
documents = SimpleDirectoryReader("resource").load_data()
index = VectorStoreIndex.from_documents(documents, embed_model=embed_model)

# 将LangChain的LLM封装成LlamaIndex的LLM
llamaLlm = LangChainLLM(llm=llm)
# 构建检索引擎
query_engine = index.as_query_engine(
    llm=llamaLlm
)
response = query_engine.query("怎么退款")
response

```

在这个过程中，实际上LlamaIndex就会从index中检索出和问题相关的条目，然后再一起发给大语言模型，最终大语言模型参考这些条目，给出答案。

另外，Query Engine只支持单次的查询。如果需要进行多轮对话，就需要使用Chat Engine。

7、Settings 全局设置

Settings是LlamaIndex中的全局设置管理器。它允许用户设置全局的配置，如LLM、索引、查询引擎、Chat Engine等。有了Settings，用户就不用在每个组价中单独配置这些参数了。

例如LlamaIndex默认都是使用的OpenAI相关的组件，在之前的示例中，我们想要使用阿里云百炼的组件，就可以使用Settings进行全局设置。

```

from llama_index.core import Settings
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.embeddings.langchain import LangchainEmbedding
from langchain_community.embeddings import DashScopeEmbeddings
from langchain_community.chat_models import ChatTongyi
from config.load_key import load_key

llm = ChatTongyi(
    model="qwen-plus",
    api_key=load_key("BAILIAN_API_KEY"),
)
# 初始化通义千问的Embedding模型
embed_model = LangchainEmbedding(

```

```
DashScopeEmbeddings(  
    dashscope_api_key=load_key("BAILIAN_API_KEY"),  
    model="text-embedding-v1" # 通义千问的Embedding模型名称 默认值  
)  
)  
  
Settings.llm = llm  
Settings.embed_model = embed_model  
  
documents = SimpleDirectoryReader("./resource").load_data()  
index = VectorStoreIndex.from_documents(documents)  
  
query_engine = index.as_query_engine()  
response = query_engine.query("怎么退款")  
response
```

在Settings中，除了可以维护llm和embed_model外，还可以维护非常多的组件。包括之间介绍过的Node Parser\Tokenizer等。

三、总结

本章主要介绍了LlamaIndex的基本组件，当然，LlamaIndex中的组件还远不止这些，各个组件提供的实现类也非常丰富。

从这里也可以看到，LlamaIndex的设计非常灵活。通过抽象出不同的组件，来实现不同的功能。同时，LlamaIndex也提供了非常多的扩展点，允许用户根据自己的需求进行扩展。

业务层面，LlamaIndex对企业典型的RAG应用场景进行了非常完善的封装。一方面通过这些封装的接口，极大的简化了RAG的实现难度。另一方面，又通过丰富的接口实现，提供了非常多的扩展点，可以应对各种不同的业务场景。